تنافسي البرامجيات

جامعة واسط - كلية علوم الحاسوب وتكنولوجيا المعلومات

https://t.me/cs_mscX

ملخص جابتر 7

الاشجار

Trees

من المصدر:

Data Structures & Algorithms in C++, 2nd, ed Michael T. Goodrich

إعداد: قناة تنافسي البرامجيات

الأشجار العامة (General Trees)

🎯 المفهوم الأساسي

من الخطي إلى الهرمي

```text
قبل: Linear (خط مستقيم)
Array:    [1] → [2] → [3] → [4]
List:     Node → Node → Node

الآن: Hierarchical (شجرة مقلوبة - هرمي)
         Root
        /  |  \
       A   B   C
      /|   |   |\
     D E   F   G H
```

تعريف الشجرة

Tree = هيكل بيانات **غير خطي** (Nonlinear) يخزن العناصر بشكل **هرمي** (Hierarchical) `1+eecs.yorku`

التعريف الرسمي

الشجرة هي مجموعة من **العقد** (Nodes) مرتبطة بعلاقة **"أب-ابن"** (Parent-Child) بحيث:

1. إذا كانت غير فارغة، يوجد **عقدة واحدة مميزة** تسمى **الجذر** (Root) ليس لها أب
2. كل عقدة أخرى لها **أب واحد فقط** (Unique Parent)

التعريف الاستدعائي **(Recursive)**

```text
Tree =
  • Empty (شجرة فارغة)
  أو
  • Root (r) + مجموعة من الأشجار الفرعية (Subtrees)
    حيث جذور هذه الأشجار هم أبناء r
```

التشبيه البشري 🌳

```text
↑ الطبيعة: الجذر في الأرض، الأوراق فوق
البرمجة: الجذر فوق، الأوراق تحت ↓ (مقلوبة!)

أمثلة حياتية:
✅ شجرة العائلة (الجد الأكبر = Root)
✅ الهيكل الإداري (المدير العام = Root)
✅ مجلدات الكمبيوتر (C:/ = Root)
✅ هيكل الكتاب (الكتاب = Root، أبناء = الفصول)
```

📖 المصطلحات الأساسية (مصطلحات العائلة)

المصطلحات من شجرة العيلة

```text
          [الجد الأكبر] ← Root
          /            \
     [الأب]           [العم] ← Siblings (أشقاء)
     /    \           /    \
[أنت] [أخوك]     [ابن عمك] [بنت عمك]
  |
[ابنك] ← Descendant (حفيد)
```

1. Root (الجذر) 👑

```text
- العقدة الوحيدة التي ليس لها أب
- نقطة البداية (الزعيم، الجد الأكبر)
- كل شجرة غير فارغة لها Root واحد فقط
```

2. Parent (الأب) و Child (الابن)

```cpp
Parent → Child

مثال:
    A ← Parent
   / \
  B   C ← Children of A
```

- **Parent**: العقدة اللي فوقك مباشرة
- **Child**: العقدة اللي تحتك مباشرة

3. Siblings (الأشقاء) 👥

```cpp
    Parent
    /  |  \
   A   B   C ← كلهم Siblings (عندهم نفس الأب)
```

التعريف: العقد التي لها نفس الأب

4. Leaves (الأوراق) أو External Nodes 🍃

```cpp
       A
      / \
     B   C
    /     \
   D       E ← Leaves (ما عندهم أولاد)

D و E = Leaves (External Nodes)
```

التعريف: العقد التي ليس لها أبناء (نهاية الخط)

5. Internal Nodes (العقد الداخلية) 🔴

```cpp
       A ← Internal
      / \
     B   C ← Internal
    /     \
   D       E

A, B, C = Internal Nodes (عندهم أولاد)
```

التعريف: العقد التي لها ابن واحد أو أكثر

6. Ancestors (الأجداد) 👴

```cpp
         A
        / \
       B   C
      /
     D
    /
   E

Ancestors of E: D, B, A (طريق الصعود للـ Root)
```

التعريف: العقدة u هي جد لـ v إذا كانت:

- نفسها v
- أو جدة لأب v

7. Descendants (الأحفاد) 👶

```cpp
        A
       / \
      B   C
     / \
    D   E

Descendants of B: D, E (كل اللي تحته)
```

التعريف: v هي حفيد لـ u إذا كانت u جداً لـ v

8. Edge (الحافة) ➖

```cpp
    A
   / \
  B   C

Edges: (A,B), (A,C)
```

التعريف: الرابط بين عقدتين u و v حيث إحداهما أب للأخرى

9. Path (المسار) 🛤️

```cpp
        A
       / \
      B   C
     /     \
    D       E

Path من A إلى D: A → B → D
Path من A إلى E: A → C → E
```

التعريف: تتابع من العقد بحيث كل عقدتين متتاليتين بينهما حافة

📊 خصائص إضافية مهمة

Depth (العمق)

```cpp
         A ← depth = 0
        / \
       B   C ← depth = 1
      /     \
     D       E ← depth = 2

depth(v) = 1 - عدد الأجداد
         = طول المسار من Root إلى v
```

Height (الارتفاع)

```cpp
         A ← height = 2
        / \
       B   C ← height = 1
      /     \
     D       E ← height = 0 (leaf)

height(v) = أطول مسار من v إلى أي ورقة تحتها
height(Tree) = height(Root)
```

Level (المستوى)

```cpp
Level 0:     A
           /   \
Level 1:  B     C
         /       \
Level 2: D        E

Level = Depth
```

🔢 الأشجار المرتبة (Ordered Trees)

التعريف

Ordered Tree = شجرة يكون فيها **ترتيب محدد** لأبناء كل عقدة

الفرق

```text
Unordered Tree (غير مرتبة):
    A               A
   / \      =      / \
  B   C           C   B
(الترتيب ما يهم، نفس الشجرة)

Ordered Tree (مرتبة):
    A               A
   / \      ≠      / \
  B   C           C   B
(الترتيب مهم، أشجار مختلفة!)
```

مثال عملي: هيكل الكتاب 📚

```text
    [الكتاب]
    /   |   \
[الفصل 1] [الفصل 2] [الفصل 3]
    |
[القسم 1.1] [القسم 1.2]

الترتيب مهم جداً!
الفصل 1 يجب أن يأتي قبل الفصل 2
→ Ordered Tree
```

متى نستخدم كل نوع؟

Ordered Tree	Unordered Tree
هيكل الكتاب	شجرة العائلة
جملة اللغة الطبيعية	المجلدات (أحياناً)
Expression Tree	Organization Chart
HTML DOM	File System

💻 Tree ADT: الواجهة البرمجية

المفهوم: Position

```cpp
// Position = مؤشر آمن للعقدة
// يسمح بالوصول بدون تعديل الهيكل

Position p;
*p;                // الوصول للعنصر
p.parent();        // الأب
p.children();      // الأبناء
p.isRoot();        // هل جذر؟
p.isExternal();    // هل ورقة؟
```

دوال Tree ADT

```cpp
class Tree {
public:
    // ═══════ معلومات عامة ═══════
    int size() const;            // عدد العقد
    bool empty() const;          // هل فارغة؟

    // ═══════ الوصول ═══════
    Position root() const;       // الجذر
    PositionList positions();    // كل العقد

    // ═══════ دوال Position ═══════
    // (داخل كلاس Position)
    Element& operator*();        // الوصول للعنصر
    Position parent();           // الأب
    PositionList children();     // الأبناء
    bool isRoot();               // هل جذر؟
    bool isExternal();           // هل ورقة؟
};
```

🏗️ التمثيل المرتبط (Linked Structure)

لماذا Linked Structure؟

```text
المشكلة:
• في Array: نحتاج معرفة عدد الأبناء مسبقاً
• في الشجرة العامة: العقدة قد يكون لها 0, 1, 2, ..., n ابن!

الحل:
• كل عقدة تحتوي على "قائمة" بأبنائها
• القائمة تتمدد حسب الحاجة
```

هيكل العقدة (Node)

```cpp
struct Node {
    // ═══════ 3 مكونات رئيسية ═══════

    1. Element element;         // البيانات المخزنة

    2. Node* parent;            // مؤشر للأب
                                // (↑ للصعود)

    3. NodeList children;       // قائمة بالأبناء
                                // (↓ للنزول)
                                // يمكن أن تكون List أو Vector
};
```

مثال بصري

```text
العقدة A تخزن:

┌──────────────────┐
│ element: 'A'     │ ← البيانات
│ parent: NULL     │ ← الأب (NULL لأنه Root)
│ children: [B,C]  │ ← قائمة الأبناء
└──────────────────┘

العقدة B تخزن:

┌──────────────────┐
│ element: 'B'     │
│ parent: → A      │ ← مؤشر للأب
│ children: [D,E]  │
└──────────────────┘
```

```
تخزن D العقدة:

┌──────────────────┐
│ element: 'D'     │
│ parent: → B      │
│ children: []     │ ← قائمة فارغة (Leaf)
└──────────────────┘
```

⚡ تحليل الأداء (Performance Analysis)

تعقيد العمليات

العملية	التعقيد	الشرح
size()	O(1)	متغير n محفوظ
empty()	O(1)	فحص بسيط
root()	O(1)	مؤشر مباشر
parent(p)	O(1)	مؤشر محفوظ في العقدة
isRoot(p)	O(1)	فحص: parent == NULL
isExternal(p)	O(1)	فحص: children.empty()
children(p)	O(cp)	cp = عدد أبناء p
positions()	O(n)	المرور على كل العقد

لماذا children() تأخذ O(cp)؟

```cpp
list<Position> children() const {
    list<Position> c;
    // نمر على كل ابن ونضيفه للقائمة
    for (auto child : v->children) {  // cp iterations
        c.push_back(Position(child));  // O(1)
    }
    return c;  // المجموع: O(cp)
}
```

مقارنة مع Array Representation

```text
Linked Structure:
✅ مرن: عدد الأبناء غير محدود
✅ سهل الإضافة/الحذف
❌ استخدام ذاكرة أكبر (pointers)

Array Representation:
✅ استخدام ذاكرة أقل
❌ غير مرن: عدد ثابت للأبناء
❌ صعب التعديل
```

💡 ملخص سريع للمراجعة

المفاهيم الأساسية

```text
✅ Tree: هيكل هرمي غير خطي
✅ Root: عقدة واحدة بلا أب
✅ Leaf: عقدة بلا أبناء
✅ Internal Node: عقدة لها أبناء
✅ Parent-Child: علاقة مباشرة
✅ Siblings: نفس الأب
✅ Ancestors: الطريق للـ Root
✅ Descendants: الطريق للأسفل
```

Linked Structure

```cpp
struct Node {
    Element data;         // البيانات
    Node* parent;         // الأب (فقط 1)
    List<Node*> children; // الأبناء (0 أو أكثر)
};
```

التعقيد الزمني

```text
O(1): size, empty, root, parent, isRoot, isExternal
O(cp): children (cp = عدد الأبناء)
O(n): positions
```

التنقل في الأشجار (Tree Traversal)

🎯 المفهوم الأساسي

ما هو Tree Traversal؟

Traversal = زيارة كل عقدة في الشجرة **مرة واحدة فقط** بطريقة نظامية ومنتظمة

التشبيه البشري 🚶

```text
تخيل عندك شجرة عائلة ضخمة وتريد تسلم على الجميع

❌ بدون نظام: راح تضيع وتنسى ناس أو تسلم مرتين
✅ مع Traversal: نظام واضح يضمن زيارة الجميع مرة واحدة

مثل: زيارة كل غرف البيت بترتيب معين
```

📏 الجزء الأول: القياسات (Measurements)

قبل ما نتنقل، لازم نعرف وين واقفين!

```text
عندنا مقياسين عكس بعض:
1. Depth (العمق) - من فوق لجوة ↓
2. Height (الارتفاع) - من جوة لفوق ↑
```

🤿 1. العمق (Depth)

التعريف

Depth = عدد **الأجداد** (Ancestors) للعقدة (بدون العقدة نفسها)

القاعدة الذهبية

```text
depth(Root) = 0 دائماً
```

الفكرة: "نظام الغوص"

```text
        Root ← depth = 0 (السطح)
       /    \
      A      B ← depth = 1 (نزلت خطوة)
     / \      \
    C   D      E ← depth = 2 (نزلت خطوتين)
   /
  F ← depth = 3 (نزلت 3 خطوات)
```

كيف نحسبه؟

طريقة بشرية: باوع فوق وعد كم "أب" فوق راسك

خوارزمية استدعائية:

```cpp
int depth(Position p) {
    if (p.isRoot()) {
        return 0;  // الجذر عمقه صفر
    }
    return 1 + depth(p.parent());  // 1 + عمق الأب
}
```

مثال تتبع

```text
        A (depth=0)
       / \
      B   C (depth=1)
     /
    D (depth=2)
   /
  E (depth=3)

depth(E):
  E ليس Root → 1 + depth(D)
  D ليس Root → 1 + depth(B)
  B ليس Root → 1 + depth(A)
  A هو Root → 0

✅ النتيجة: 1 + 1 + 1 + 0 = 3
```

التعقيد الزمني

```text
Best Case: O(1) - إذا كانت العقدة هي Root
Average: O(d) - حيث d عمق العقدة
Worst Case: O(n) - إذا كانت الشجرة خط مستقيم
```

🧮 2. الارتفاع (Height)

التعريف

Height = أطول مسار من العقدة إلى أي **ورقة** (Leaf) تحتها

القاعدة الذهبية

```text
height(Leaf) = 0 دائماً
height(Tree) = height(Root)
```

الفكرة: "نظام البنايات"

```text
        Root ← height = 3 (أعلى طابق)
       /    \
      A      B ← height = 2
     / \      \
    C   D      E ← height = 1
   /
  F ← height = 0 (ورقة - القاع)
```

كيف نحسبه؟

طريقة بشرية: باوع جوة وشوف أطول طريق للقاع

خوارزمية استدعائية:

```cpp
int height(Position p) {
    if (p.isExternal()) {  // ورقة؟
        return 0;
    }

    int maxHeight = 0;
    for (auto child : p.children()) {
        int h = height(child);
        if (h > maxHeight) {
            maxHeight = h;
        }
    }

    return 1 + maxHeight;  // 1 + أقصى ارتفاع للأبناء
}
```

مثال تتبع

```text
        A
       / \
      B   C
     /     \
    D       E
   /
  F

height(A):
  A ليست Leaf
  أبناء A: [B, C]

  height(B):
    B ليست Leaf
    أبناء B: [D]
    height(D):
      D ليست Leaf
      أبناء D: [F]
      height(F): F 0 → ورقة
      height(D) = 1 + 0 = 1
    height(B) = 1 + 1 = 2
```

```
height(C):
  C ليست Leaf
  أبناء C: [E]
  height(E): E 0 → ورقة
  height(C) = 1 + 0 = 1

max(2, 1) = 2
height(A) = 1 + 2 = 3 ✅
```

📊 مقارنة Depth vs Height

الخاصية	Depth	Height
الاتجاه	من فوق لجوة ↓	من جوة لفوق ↑
نقطة البداية	Root = 0	Leaf = 0
الحساب	عد الأجداد	أطول مسار للورقة
التشبيه	الغوص في البحر	البناية
السؤال	كم واحد فوقي؟	كم واحد تحتي؟

مثال توضيحي كامل

```text
        A (d=0, h=3)
       / \
      B   C (d=1, h=1)
     /     \
    D       E (d=2, h=0) ← Leaf
   /
  F (d=3, h=0) ← Leaf

d = depth, h = height

الملاحظة:
• Leaves دائماً height=0
• Root دائماً depth=0
• height(Tree) = أقصى depth في الشجرة
```

🧭 الجزء الثاني: خوارزميات التنقل

السؤال الأساسي

```text
عندي شجرة، وأريد أزور كل عقدة مرة واحدة
بأي ترتيب أزورهم؟

الجواب: عندنا استراتيجيتين رئيسيتين
```

📦 1. Preorder Traversal (الترتيب المسبق)

التعريف

Preorder = أزور الأب أولاً، ثم أزور الأبناء

القاعدة

```text
Visit (الأب) → Visit (الابن 1) → Visit (الابن 2) → ...
```

التشبيه: "المدير المتسلط"

```text
"المدير يقول: "أنا أخلص شغلي أولاً، بعدين جهالي يشتغلون

"أو: "أقرأ عنوان الفصل أولاً، بعدين أقرأ المحتوى
```

الخوارزمية

```cpp
void preorder(Position p) {
    visit(p);  // 1. زر العقدة الحالية أولاً ✅

    // 2. بعدين زر كل الأبناء
    for (auto child : p.children()) {
        preorder(child);  // استدعاء تكراري
    }
}

// دالة الزيارة (مثال: طباعة)
void visit(Position p) {
    cout << *p << " ";
}
```

مثال تتبع

```text
        A
       /|\
      B C D
     /|   |
    E F   G

الترتيب: A B E F C D G

الخطوات:
1. visit(A) → طباعة A
2. preorder(B)
   - visit(B) → طباعة B
   - preorder(E)
     - visit(E) → طباعة E
   - preorder(F)
     - visit(F) → طباعة F
3. preorder(C)
   - visit(C) → طباعة C
4. preorder(D)
   - visit(D) → طباعة D
   - preorder(G)
     - visit(G) → طباعة G

النتيجة: A B E F C D G ✅
```

الملاحظة المهمة

```text
في Preorder:
• الأب يُطبع قبل كل أبنائه
• كل شجرة فرعية تُطبع كاملة قبل الانتقال للتالية
```

استخدامات Preorder

```text
✅ طباعة جدول المحتويات (الفصول قبل الأقسام)
✅ عمل نسخة من الشجرة
✅ Expression Tree (العملية قبل المعاملات)
✅ Prefix Notation
```

2. Postorder Traversal (الترتيب البعدي) 👷

التعريف

Postorder = أزور **الأبناء أولاً**، ثم أزور **الأب**

القاعدة

```text
Visit (الابن 1) → Visit (الابن 2) → ... → Visit (الأب)
```

التشبيه: "الموظف المظلوم"

```text
"الأب يقول: "ما أقدر أخلص شغلي إلا لما كل جهالي يخلصون ويسلموني التقارير

"أو: "لازم أجمع كل قطع الليجو الصغيرة قبل ما أحصل على اللعبة الكاملة
```

الخوارزمية

```cpp
void postorder(Position p) {
    // 1. زر كل الأبناء أولاً
    for (auto child : p.children()) {
        postorder(child);  // استدعاء تكراري
    }

    visit(p);  // 2. بعدين زر العقدة الحالية ✅
}
```

الملاحظة المهمة

```text
في Postorder:
• الأب يُطبع بعد كل أبنائه
• الأوراق (Leaves) تُطبع أولاً
• Root يُطبع أخيراً
```

مثال تتبع

```text
        A
       /|\
      B C D
     /|   |
    E F   G

الترتيب: E F B C G D A

الخطوات:
1. postorder(A)
   - postorder(B)
     - postorder(E)
       - E ورقة → visit(E) → طباعة E
     - postorder(F)
       - F ورقة → visit(F) → طباعة F
     - visit(B) → طباعة B
   - postorder(C)
     - C ورقة → visit(C) → طباعة C
   - postorder(D)
     - postorder(G)
       - G ورقة → visit(G) → طباعة G
     - visit(D) → طباعة D
   - visit(A) → طباعة A

النتيجة: E F B C G D A ✅
```

استخدامات Postorder

```text
✅ حساب مساحة المجلدات
✅ حذف الشجرة (احذف الأبناء قبل الأب)
✅ تقييم Expression Tree
✅ Postfix Notation
✅ حساب خصائص تعتمد على الأبناء
```

🆚 مقارنة Preorder vs Postorder

الخاصية	Preorder	Postorder
ترتيب الزيارة	الأب → الأبناء	الأبناء → الأب
الأب يُطبع	أولاً	أخيراً
الأوراق تُطبع	في المنتصف	أولاً
Root يُطبع	أولاً	أخيراً
التشبيه	المدير المتسلط	الموظف المظلوم
الاستخدام	جدول المحتويات	حساب المساحات

مثال مقارنة

```text
        A
       / \
      B   C
     /
    D

Preorder:  A B D C
           ↑ (الأب أولاً)

Postorder: D B C A
                 ↑ (الأب أخيراً)
```

⚡ تحليل الأداء

التعقيد الزمني

العملية	التعقيد	الشرح
depth(p)	O(dp)	dp = عمق p
	Worst: O(n)	شجرة خطية
height(p)	O(n)	تزور كل عقدة مرة واحدة
()preorder	O(n)	تزور كل عقدة مرة واحدة
()postorder	O(n)	تزور كل عقدة مرة واحدة

لماذا Traversals دائماً O(n)؟

```text
السبب: نزور كل عقدة من الـ n عقدة مرة واحدة فقط

في كل زيارة:
• نطبع العنصر: O(1)
• نستدعي الدالة للأبناء: عدد الاستدعاءات = عدد الأبناء

مجموع الاستدعاءات = مجموع الأبناء لكل العقد
                   = n - 1 (عدد الحواف)
                   = O(n)

النتيجة: O(n) ✅
```

🌊 3. تنقلات أخرى (إشارة سريعة)

Breadth-First Traversal (BFS)

```text
الفكرة: زر العقد حسب المستوى (Level)

        A (level 0)
       /|\
      B C D (level 1)
     /|   |
    E F   G (level 2)

الترتيب: A → B C D → E F G

التنفيذ: باستخدام Queue (طابور)

الاستخدام:
✅ أقصر مسار
✅ Level Order Traversal
✅ أقرب عقدة
```

Inorder Traversal

```text
⚠️ خاص بالأشجار الثنائية فقط!

الترتيب: Left → Parent → Right

مثال:
      B
     / \
    A   C

الترتيب: A B C

الاستخدام: Binary Search Tree (BST)
```

📝 ملخص سريع للمراجعة

القياسات

```text
Depth (العمق):
• من Root للأسفل
• depth(Root) = 0
• عد الأجداد
• O(d) حيث d عمق العقدة

Height (الارتفاع):
• من Leaves للأعلى
• height(Leaf) = 0
• أطول مسار للورقة
• O(n) لكل الشجرة
```

التنقلات

```text
Preorder (مُسبق):
• الأب قبل الأبناء
• visit → children
• استخدام: جدول محتويات
• O(n)

Postorder (بَعدي):
• الأبناء قبل الأب
• children → visit
• استخدام: حساب مساحات
• O(n)
```

أسئلة Tracing

السؤال: ما ترتيب Preorder و Postorder للشجرة التالية؟

```text
      A
     /|\
    B C D
   /    |
  E     F
```

الحل:

```text
Preorder: A B E C D F
(الأب أولاً)
A → B → E → C → D → F

Postorder: E B C F D A
(الأبناء أولاً)
E → B → C → F → D → A
```

الأشجار الثنائية (Binary Trees)

🎯 المفهوم الأساسي

ما هي الشجرة الثنائية؟

Binary Tree = شجرة مرتبة كل عقدة فيها لها **طفلان على الأكثر** (0، 1، أو 2)

القاعدة الذهبية

```text
كل طفل له "هوية محددة":
• إما Left Child (طفل أيسر)
• أو Right Child (طفل أيمن)

⚠️ حتى لو عندك طفل واحد، لازم تحدد: يسار أو يمين؟
السبب: المكان يغير المعنى في الخوارزميات (مثل BST)
```

📋 أنواع الأشجار الثنائية

1. Binary Tree (عامة)

```text
        A
       / \
      B   C
     /
    D

• أي عقدة: 0، 1، أو 2 طفل
• B عندها طفل واحد (D) على اليسار
```

2. Proper / Full Binary Tree (السليمة)

```text
        A
       / \
      B   C
     / \
    D   E

"القاعدة: "يا 0 يا 2 - ممنوع 1
```

```
✅ كل عقدة إما:
   • ليس لها أبناء (Leaf)
   • أو لها ابنان بالضبط

❌ ممنوع عقدة لها ابن واحد فقط
```

التعريف الاستدعائي (Recursive)

شجرة تنائيه = واحد من الاتي:

1. شجرة فارغة(Empty Tree)
2. جذر + (Root) شجرة يسرى + شجرة يمنى حيث كل واحدة منهما شجرة ثنائية

🔪 الواجهة البرمجية (Binary Tree ADT)

عمليات Position

```cpp
// العمليات الخاصة بالأشجار الثنائية
Position left()       // الطفل الأيسر
Position right()      // الطفل الأيمن

// العمليات العامة من General Trees
Position parent()     // الأب
bool isRoot()        // هل هي الجذر؟
bool isExternal()    // هل هي ورقة؟
E& operator*()       // الوصول للعنصر
```

عمليات Tree

```cpp
int size()                  // عدد العقد
bool empty()               // هل فارغة؟
Position root()            // الجذر
PositionList positions()   // قائمة كل العقد
```

الفرق عن General Trees

```text
General Tree:
  list<Position> children()  // قائمة كل الأبناء

Binary Tree:
  Position left()            // اليسار مباشرة
  Position right()           // اليمين مباشرة

الفائدة: أسرع وأوضح - بدون تكرار على قائمة
```

📐 الخصائص الرياضية (Properties)

1. علاقة الارتفاع بعدد العقد

```text
لأي شجرة ثنائية غير فارغة:

h + 1 ≤ n ≤ 2^(h+1) - 1

حيث:
• h = الارتفاع
• n = عدد العقد

الحد الأدنى: شجرة خطية (مثل قائمة)
الحد الأقصى: شجرة ممتلئة تماماً
```

مثال توضيحي

```text
h = 2 (ارتفاع 2)

الحد الأدنى: n = h + 1 = 3 عقد
      A
     /
    B
   /
  C

الحد الأقصى: n = 2^3 - 1 = 7 عقد
       A
      / \
     B   C
    / \ / \
   D E F G
```

2. قاعدة الأوراق (في Proper Binary Tree)

```text
n_e = n_i + 1

حيث:
• n_e = عدد العقد الخارجية (External Nodes / Leaves)
• n_i = عدد العقد الداخلية (Internal Nodes)

مثال:
        A
       / \
      B   C
     / \
    D   E

n_i = 2 (A, B)
n_e = 3 (C, D, E)

1 + 2 = 3 :التحقق ✅
```

3. عقد المستوى

المستوى d يحتوي على الأكثر حيث عدد المستوى مرفوع لأس 2 يساوي عدد العقد كما في المثال:

d=0: $2^0 = 1$ عقدة (Root)

d=1: $2^1 = 2$ عقد

d=2: $2^2 = 4$ عقد

d=3: $2^3 = 8$ عقد

التنفيذ الأول: Linked Structure

مميزات Linked Structure

✅ المرونة: يمكن إضافة/حذف عقد بسهولة

✅ الكفاءة: مساحة O(n) بالضبط

✅ السرعة: معظم العمليات تكون O(1)

العمليات:

اسم العملية	التعقيد الزمني لها
left(), right(), parent()	O(1)
addLeft(), addRight()	O(1)
expandExternal()	O(1)
removeAboveExternal()	O(1)

التشبيه البشري

```text
كل عقدة = صندوق بـ 4 جيوب:

┌──────────────────┐
│  العنصر: 'A'     │ ← البيانات
├──────────────────┤
│  ↑ Parent        │ ← عنوان الأب
│  ← Left          │ ← عنوان الابن الأيسر
│  → Right         │ ← عنوان الابن الأيمن
└──────────────────┘
```

📊 التنفيذ الثاني: Array / Vector-Based

فكرة Level Numbering

```text
ترقيم العقد حسب المستوى:

        1 (Root)
       / \
      2   3
     / \ / \
    4 5 6 7

القاعدة:
• الجذر: index = 1
• لأي عقدة بـ index = i:
  - الابن الأيسر: 2 - i
  - الابن الأيمن: 2 - i + 1
  - الأب: i/2 (قسمة صحيحة)
```

مثال عملي

```text
بناء الشجرة:

        A(1)
       /    \
     B(2)   C(3)
    /   \
  D(4) E(5)

المصفوفة:
Index: [0] [1] [2] [3] [4] [5] [6] [7]
Value: [ ] [A] [B] [C] [D] [E] [ ] [ ]

العمليات:
• left(1) = tree[2] = B ✅
• right(1) = tree[3] = C ✅
• parent(4) = tree[4/2] = tree[2] = B ✅
• left(2) = tree[4] = D ✅
• right(2) = tree[5] = E ✅
```

المشكلة الكبيرة ⚠️

```text
شجرة غير متوازنة:

       A(1)
          \
           B(3)
              \
               C(7)
                  \
                   D(15)

المصفوفة المطلوبة:
Index: 1   2   3   4   5   6   7 ... 15
Value: A   ×   B   ×   ×   ×   C ... D

          ↑ خانات فارغة ↑

الحجم: O(2^h) حيث h الارتفاع
هدر كبير في الذاكرة! ☠️
```

المقارنة

الخاصية	Linked	Array
المساحة	O(n) دائماً	O(2^h) أسوأ حالة
الوصول للأب	O(1)	O(1)
الوصول للأبناء	O(1)	O(1)
المرونة	عالية	محدودة
الاستخدام	الأشجار العامة	الأشجار الممتلئة فقط

متى نستخدم Array؟

```text
✅ مناسب لـ:
• Complete Binary Tree (ممتلئة)
• Heap (الكومة)
• Priority Queue

❌ غير مناسب لـ:
• أشجار غير متوازنة
• أشجار البحث العادية (BST)
```

🧘 خوارزميات التنقل (Traversals)

1. Preorder (الترتيب المسبق)

```text
الترتيب: Root → Left → Right

void preorder(Position p) {
    if (p.v == NULL) return;

    visit(p);               // 1. زر الجذر
    preorder(p.left());     // 2. زر اليسار
    preorder(p.right());    // 3. زر اليمين
}
```

مثال Preorder

```text
        A
       / \
      B   C
     / \
    D   E

الترتيب: A → B → D → E → C

الخطوات:
1. visit(A)
2. preorder(B)
   - visit(B)
   - preorder(D) → visit(D)
   - preorder(E) → visit(E)
3. preorder(C)
   - visit(C)

النتيجة: A B D E C ✅
```

2. Postorder (الترتيب البعدي)

```text
الترتيب: Left → Right → Root

void postorder(Position p) {
    if (p.v == NULL) return;

    postorder(p.left());   // 1. زر اليسار
    postorder(p.right());  // 2. زر اليمين
    visit(p);              // 3. زر الجذر
}
```

مثال Postorder

```text
       A
      / \
     B   C
    / \
   D   E

الترتيب: D → E → B → C → A

الخطوات:
1. postorder(A)
   - postorder(B)
     - postorder(D) → visit(D)
     - postorder(E) → visit(E)
     - visit(B)
   - postorder(C)
     - visit(C)
   - visit(A)

النتيجة: D E B C A ✅
```

3. Inorder (الترتيب البيني) ⭐ جديد

```text
الترتيب: Left → Root → Right

void inorder(Position p) {
    if (p.v == NULL) return;

    inorder(p.left());      // 1. زر اليسار
    visit(p);               // 2. زر الجذر
    inorder(p.right());     // 3. زر اليمين
}
```

مثال Inorder

```text
        A
       / \
      B   C
     / \
    D   E

الترتيب: D → B → E → A → C

الخطوات:
1. inorder(A)
   - inorder(B)
     - inorder(D) → visit(D)
     - visit(B)
     - inorder(E) → visit(E)
   - visit(A)
   - inorder(C)
     - visit(C)

النتيجة: D B E A C ✅
```

لماذا Inorder مهم؟ 🔥

```text
في Binary Search Tree (BST):

        50
       /  \
     30    70
    / \    / \
   20 40  60 80

Inorder: 20 30 40 50 60 70 80

✨ النتيجة: الأرقام مرتبة تصاعدياً!

هذا هو "سر الترتيب" في BST
```

4. Euler Tour Traversal (جولة أويلر)

```text
المفهوم: "الحارس الذي يمشي حول الشجرة"

كل عقدة تُزار 3 مرات:
1. من اليسار (قبل دخول اليسار)
2. من الأسفل (بين اليسار واليمين)
3. من اليمين (بعد الخروج من اليمين)
```

رسم Euler Tour

```text
        A
       / \
      B   C
     /
    D

المسار (بالأرقام):

        A₁—A₂—A₃
       /         \
    B₁—B₂—B₃      C₁—C₂—C₃
   /
D₁—D₂—D₃

الزيارات بالترتيب:
A₁ B₁ D₁ D₂ D₃ B₂ B₃ A₂ C₁ C₂ C₃ A₃
```

```
اختيار موعد الطباعة:
• 1" طباعة في": Preorder → A B D C
• 2" طباعة في": Inorder → D B A C
• 3" طباعة في": Postorder → D B C A
```

📊 جدول مقارنة Traversals

الخوارزمية	الترتيب	المثال	الاستخدام
Preorder	Root-Left-Right	A B D E C	نسخ الشجرة، Prefix
Inorder	Left-Root-Right	D B E A C	BST مرتب، Infix
Postorder	Left-Right-Root	D E B C A	حذف، حساب، Postfix
Euler Tour	3 زيارات لكل عقدة	-	Expression Trees

🎨 Template Function Pattern

المفهوم البشري

```text
"الهيكل العظمي للكود"

كل مرة Traversal بدل ما تكتب كود:

❌ كود Preorder للطباعة
❌ كود Preorder للحساب
❌ كود Preorder للنسخ
...

✅ اكتب هيكل واحد (Template)
✅ خلي دوال فارغة
✅ أي حد يرث الكلاس ويعبي الدوال
```

الفائدة:

- ✅ تقليل التكرار في الكود(DRY)
- ✅ سهولة إضافة وظائف جديدة
- ✅ فصل "القراءة والمشي على العقد" عن "الوظيفة او الدوال"
- ✅ نمط تصميم قائم على الكائنات قوي

🔄 تحويل General Tree إلى Binary Tree

المشكلة

```text
الشجرة العامة: العقدة لها عدد غير محدود من الأبناء

        A
      / | \
     B  C  D
    /|  |
   E F  G

Binary Tree: العقدة لها طفلان فقط (Left, Right)

كيف نمثل هذه الشجرة المعقدة في ثنائية؟
```

الحل: "خدعة الابن والأخ"

```text
القاعدة:
• Left Child = الابن الأول (First Child)
• Right Child = الأخ التالي (Next Sibling)

يعني:
• اليسار يوديك لابنك
• اليمين يوديك لأخوك
```

مثال التحويل

```text
General Tree:

        A
      / | \
     B  C  D
    /|
   E F

الخطوات:
1. A: أول ابن = B → left(A) = B
2. B: أخوه = C → right(B) = C
3. C: أخوه = D → right(C) = D
4. B: أول ابن = E → left(B) = E
5. E: أخوه = F → right(E) = F
```

```
Binary Tree الناتجة:

       A
      /
     B
    / \
   E   C
    \   \
     F   D

قراءتها:
• left = ابن
• right = أخ
```

الفائدة

```text
✅ تمثيل أي شجرة معقدة بمؤشرين فقط
✅ توفير الذاكرة
✅ استخدام خوارزميات Binary Trees على General Trees
✅ تبسيط التنفيذ
```

Tracing

السؤال: ما ترتيب Inorder للشجرة؟

```text
      A
     / \
    B   C
   / \
  D   E
```

الحل: D B E A C

```text
الخطوات:
1. inorder(A)
   - inorder(B)
     - inorder(D) → D
     - B
     - inorder(E) → E
   - A
   - inorder(C) → C

النتيجة: D B E A C ✅
```

True/False

```text
1. كل Binary Tree هي Proper Tree.
   ❌ False (Proper فقط 2 أو 0 :شرط إضافي)

2. Inorder خاص بالأشجار الثنائية فقط.
   ✅ True (لأنه يحتاج left/right محددان)

3. Array representation مناسب لكل الأشجار.
   ❌ False (فقط للممتلئة/المتوازنة)

4. في Proper Tree: $n_e > n_i$ دائماً.
   ✅ True ($n_e = n_i + 1$)

5. Euler Tour يمكنه توليد الثلاث traversals.
   ✅ True (حسب موضع الطباعة)
```